OGC2026 Technical Report

Abstract. The challenge asks for a schedule and a physical layout at once: every block must be given a bay, an entry time and a collision-free pose, under a crane that descends vertically and therefore forbids placements that a purely two-dimensional packing would allow. We observe that the objective $w_1 Z_1 + w_2 Z_2 + w_3 Z_3$ reads only the pair (bay, entry time) — coordinates and orientation appear nowhere in it — so geometry is a feasibility constraint that governs how much scheduling freedom exists, not a term to be optimised directly. The share of the objective carried by tardiness ranges from 0% to 97.4% across the final instances with nothing in between, so we deliberately avoid any density threshold or instance classification. Instead a single portfolio of six operators — beam construction, evolutionary re-growth, two repair passes, a CP-SAT bay reassignment and an exact bay repacking — competes for the time budget on measured payoff per second, with slice sizing driven by completion rather than by payoff. All hot loops are C++ extensions. Our main contributions for the final round are an exact contact upper bound that prunes 40.8% of the candidate cells in the construction while provably returning the identical solution (1.85× faster on the sparsest instance), a 10.5× faster conflict-graph build proved to produce a bit-identical graph, and a position-independent geometric predicate that removed a floating-point inconsistency from the packing test.

1 Introduction

1.1 Challenge Problem & Main Difficulties

Three difficulties shaped every design decision we made.

The objective and the constraints look at different variables. The objective is a weighted sum of tardiness Z_1, workload imbalance Z_2 and preference score Z_3, and each of the three is a function of *which bay* a block occupies and *when* it enters. The block's x, y and orientation do not appear. Geometry enters only through feasibility: a layout that cannot accommodate a block forces its entry later, which is how packing quality is converted into objective value. Treating tightness as an objective term is therefore a modelling error, and one we made early — weighting contact more heavily does not buy a better schedule, it buys a denser layout whose scheduling consequences are unmeasured.

The crane makes the packing three-dimensional. A block is lowered vertically, so a placement is legal only if every layer of the descending block clears every layer of what is already present along the whole descent. A one-layer neighbour and a two-layer neighbour occupy the same floor cells but ration space entirely differently: the resource that is actually scarce is *height*, not occupancy. Our

contact measure is computed per layer for this reason, so that a block standing beside a shorter one earns nothing for the layers that face open air.

Instances are not on a spectrum. Measured on the final instances, the tardiness share of the objective is exactly 0% on the sparsest and 97.4% on the densest, with no instance in between. Where Z_1 is zero, entry times are free and the problem is a pure assignment problem dominated by preference; where the yard is saturated, entry times are everything. Any hand-drawn density threshold has to guess where the switch lies, and the observed distribution offers no evidence about it. We therefore built a scheduler that *measures* which operator is paying rather than one that classifies the instance.

1.2 Contributions

1. **A gate-free, measured-payoff operator portfolio.** Six heterogeneous operators share one budget. Selection is by realised objective gain per second; slice sizing is a separate question answered by completion. No branch anywhere in the algorithm tests instance size, density or index.
2. **An exact contact upper bound for the construction.** We bound the achievable contact of a candidate cell by an occupancy rectangle query, which lets the beam discard cells that provably cannot win. It removes 40.8% of scored cells on the sparsest final instance and returns a bit-identical solution (1.85×, 1.66× and 1.09× faster on three instances of very different density).
3. **A 10.5× faster conflict-graph build with a proved-identical graph.** Two space–time columns can conflict only while both are present, so an entry-sorted sweep with an early break never visits 83% of the pairs. Because the transformation cannot change which pairs conflict, the edge count is the acceptance test, not the clock.
4. **Offset-relative geometry.** Replacing absolute-coordinate overlap tests with origin-outline comparisons plus an offset made the packing predicate position-independent and removed a floating-point disagreement that had been misdiagnosed as a hashing problem.
5. **A verification discipline that we report as a result.** Every acceleration in this system is accepted or rejected by an invariant that must match exactly — an edge count, or a digest over every returned placement — and only then is a clock read. Section 2.3 lists what this discipline killed.

2 Algorithm Description

2.1 Overall Algorithm Framework

The entry point reserves $\min(0.2T, 40\,\mathrm{s})$ of the time limit for a final preference-repair pass and spends the rest on four independent worker processes. Each worker seeds a pool with a cheap always-feasible sequential schedule — the single guarantee that the algorithm never returns nothing, which is what allowed every

other fallback path to be deleted — and then runs one operator loop until its budget is exhausted. The workers differ in random seed and in their starting offset into a list of six diversification axes. The best full objective over the workers is taken, polished, and returned.

Feasibility is enforced by construction inside the operators and *re-checked* outside them: the single acceptance function re-runs the official feasibility checker on any solution that beats the incumbent and returns $+\infty$ if it fails. This bounds the risk of an over-permissive packer — an operator may be wrong about feasibility, but a wrong answer cannot be accepted.

The operator portfolio. Six operators are available: a contact-beam construction; an evolutionary re-growth with crossover and path-relinking over (bay, dispatch-order) anchors, decoded by the same beam; a workload-balance repair; a preference repair; a CP-SAT bay reassignment; and an exact bay repacking described in Section 3.

Selection and sizing are different questions. Each operator receives one probe to establish a rate, after which the budget goes to the operator with the highest realised objective gain per second, with 15% exploration so a slow starter can recover. Slice *size* is deliberately not driven by that same signal. Sizing on payoff death-spirals: a construction that returns a perfectly good solution which merely fails to beat the incumbent scores zero gain, shrinks, and then can no longer finish at all — we measured fourteen calls returning four solutions and the objective moving from 48 840 to 56 841. Size is instead a completion question: a search operator that returned nothing was starved and is given more; one that returned anything fitted; a repair pass, which always completes, is given what it actually used.

Asymmetric opening slices. Search operators open on a fifth of the budget and repair passes on budget$/2n$. Both symmetric alternatives are worse and were measured: a flat fifth for all six operators spends 1.2× the entire budget before anything is tried twice (on one dense instance the first construction produced the best solution of the run in 33 s and the remaining 267 s went on first probes), while giving every operator the small derived share starves the constructions, degrading Z_1, Z_2 and Z_3 simultaneously.

2.2 Key Ideas that Succeeded

Contact beam with per-layer tightness. The construction is a beam search over dispatch order. Each state places the next block at the position minimising a weighted sum of negative contact, a positional sweep term and a preference penalty; survivors are completed by a rollout and ranked on the exact objective. Contact is accumulated per layer and divided by the layer count, so a neighbour of matching height scores what a flat count would give and a mismatched one only a fraction. The flat count, summed over layers without normalisation, inflates tightness by 1.35–1.95× against a fixed tardiness weight and was measurably harmful.

An exact contact upper bound. Profiling showed that 86% of scored cells were evaluated *after* the best cell of that scan had already been found. Our first attempt bounded the positional term and fired zero times in 112.9 million cells: the score is contact-dominated by more than an order of magnitude (a positional range of ≈ 2.3 against a contact range of ≈ 60), so a bound that knows only the sweep coordinate is never competitive.

Bounding the contact itself inverts this. The contact of a placement counts, for each occupied footprint cell, its four neighbours: a neighbour outside the bay scores one, a neighbour inside the same footprint scores nothing, and an occupied neighbour scores its weight. Every occupied cell that can contribute therefore lies inside the footprint bounding box dilated by one, and can be counted at most once per direction, giving

$$\mathrm{ct}(i_x, i_y) \;\le\; W(i_x, i_y) \;+\; 4\, w_{\max} \left| \mathcal{O} \cap B^{+1}(i_x, i_y) \right|, \tag{1}$$

where W counts the cell–direction pairs pointing out of the bay, $\mathcal{O}$ is the occupied set and B^{+1} the dilated bounding box. A two-dimensional occupancy prefix sum — built once per (bay, time window) at the same asymptotic cost as the occupancy map it accompanies — answers the rectangle in four reads, after which a cell whose *best possible* score cannot beat the incumbent is skipped without any geometric test.

The bound is applied only where it is proved: the layered contact variant, which counts against per-layer occupancy that a flat prefix sum does not dominate, is excluded, as are the auxiliary scoring terms whose sign we do not establish.

A conflict-graph build that watches its own clock. The repacking operator (Section 3) begins with an $O(n_{\mathrm{col}}^2)$ conflict graph that cannot be interrupted. We made it $10.5\times$ faster by exploiting the fact that two columns can conflict only while both are present: sorting by entry time and breaking the inner loop when the later column's entry passes the earlier one's exit skips 83% of the pairs without visiting them, and the fields the filter reads were moved into flat arrays so the scan streams six sequential arrays instead of thirty thousand scattered structures. Neither change can alter which pairs conflict, so the edge count must come out identical, and *that* was the acceptance test. In production the same configuration now builds in 10.7–14.2 s where it previously cost ≈ 90 s. The build additionally projects its own completion every 256 rows from the most recent chunk and abandons a configuration it cannot finish, stepping down to a cheaper one rather than losing the call.

Offset-relative geometry. Memoising the conflict predicate initially produced 36 extra edges. We assumed key aliasing; dumping the disagreements showed the predicate itself was position-dependent through floating-point rounding. Comparing origin outlines plus an offset makes the verdict independent of absolute position, which fixed the memo by fixing the predicate.

2.3 Key Ideas that Failed

We report these because each was refuted by a measurement we had committed to in advance, and because two of them looked obviously correct.

A free-column bitset. Intended to accelerate the packing search, it was proved to produce identical results and measured 1.6–1.9× *slower*: the saving scales with columns per block while the cost scales with edges, and we had assumed the wrong line was hot.

A positional dominance bound. Exact, and verified to change no solution, but it fired zero times in 112.9 million cells for the reason given above. Its first "identity passed" verdict was meaningless — the branch had never executed — which is why our later bound was verified per cell instead.

Firing rate as a proxy for speedup. We predicted the contact bound would be *net negative* on the densest instance, where it prunes only 1.3% of cells while paying the full prefix-sum cost, and marginal on another at 3.4%. Both predictions were wrong in the same direction (1.09× and 1.66×). Cell counts are not seconds: most cells are rejected by a cheap bitmap probe, whereas the cells the bound removes are precisely those that would have reached the exact geometric test.

Tightness as an objective term. Raising the contact weight does not promote the intended ordering; it deletes contact and fragments the packing. Turning candidate contact off entirely is the best configuration we ever measured on the sparsest instance and the worst on a dense one (+25.9%), which is what led us to the per-layer, height-aware formulation.

2.4 Further Improvement Plan

Attempts versus attempt length. On the sparsest instance the reported objective is literally the minimum of the four solutions the repacking operator produced in the run, drawn from a distribution spanning 13 765. Under a min-of-N objective, halving each attempt to double N is profitable whenever per-attempt quality degrades more slowly than the extra draw helps. The 10.5× build speedup is currently spent as a *longer search inside the same four attempts*, and we are measuring the objective-against-budget curve at equal total spend to decide whether to re-split it.

A cumulative rather than per-call operator time bound. Our current per-call slice cap costs the densest instance measurably; bounding cumulative operator time instead is untested.

Removing a remaining conditional. One experimental branch still ships defaulting to the previous behaviour. It will be made unconditional or deleted; a flag that selects behaviour is exactly the kind of hidden gate this design otherwise avoids.

3 Implementation Details

Software environment. Python 3.12 orchestrates; two C++17 extension modules built with `pybind11` and OpenMP hold every hot loop. CP-SAT (OR-Tools) is used for one operator and the algorithm degrades gracefully to the remaining five if it is unavailable. Build instructions accompany the source.

Division of labour between the languages. Python holds only control flow: the scheduler, operator definitions and the acceptance test. Every geometric predicate, every candidate scan and both search engines are C++. The construction engine parallelises over beam states with OpenMP; the top-level portfolio parallelises over worker processes, and the two are never nested (BLAS-style thread pools are pinned to one thread inside workers).

The repacking operator. This operator lifts *every* block out of the most contested bay, adds the outside blocks that might want to enter it, and hands the whole set to an exact maximum-weight set-packing solver over space–time columns, weighted directly in objective units. A call has two phases of completely different character: an uninterruptible $O(n_{\mathrm{col}}^2)$ conflict-graph build, and a deadline-honouring local search. The solver therefore accepts a total-time bound and enforces it against its *own measured* build cost rather than a predicted one; an earlier version subtracted a predicted build from the search budget as well, double-charging it. The configuration ladder is expressed as fractions of each instance's own entry-time ceiling, so the window widths are derived from the instance rather than fitted to it.

Verification instrumentation. The engines carry always-available counters (cells visited, exact tests, cheap rejections, cells scored after the best was found, cells skipped by the bound) enabled by an environment variable, and a dedicated soundness mode that scores every cell the bound wanted to skip and counts the ones that would have won. On three instances of very different density that count was zero out of 45.9, 11.6 and 9.3 million skipped cells.

4 Computational Results

Table 1 reports the six final-round instances at their respective time budgets, decomposed into the three objective components. All solutions pass the official feasibility checker.

The decomposition shows directly why no single mechanism can carry the whole set: Z_1 is exactly zero on P1–P3, where the objective is dominated by preference and imbalance, and reaches 97.4% of the weighted objective on P6.

Table 2 tracks the effect of the successive changes on the two instances where they are separable. Two cautions belong with these numbers and we state them rather than smoothing them over. First, P6's run-to-run spread is approximately 112 000 on unchanged code, so differences smaller than that are noise. Second,

Table 1. Final-round instances: objective decomposition. T is the per-instance time budget.

Instance	T (s)	Z_1	Z_2	Z_3	Objective
P1	60	0	33	73	11 280
P2	120	0	3 342	90	31 368
P3	240	0	3 445	465	86 975
P4	480	88	2 561	3 933	1 781 181
P5	600	661	1 536	3 015	9 224 860
P6	900	4 301	2 164	5 827	29 566 129

Table 2. Effect of the successive changes. Differences inside the stated run-to-run spreads are not claimed as improvements.

Configuration	P3	P6
Portfolio without the repacking operator	96 990	—
+ exact bay repacking	80 795	29 651 637
+ faster build, derived configuration ladder, contact bound	86 975	29 566 129

P3's reported objective is a minimum over the repacking operator's independent attempts within a run, so single P3 numbers carry an order-statistic spread of roughly 10 000; the table gives the best clean measurement and we do not claim differences inside that band.

Speed results, measured at identical work. A fixed-time A/B cannot evaluate an accelerated search: both arms consume the same budget, so the faster arm searches further and legitimately reaches a different solution. We therefore replay a single construction call with identical arguments and no deadline, and compare a digest over every returned placement.

Finally, the conflict-graph build was accepted only after the edge count was confirmed unchanged across every configuration tested; the resulting speedup is 10.5× in isolation and appears in production as a build cost of 10.7–14.2 s where the same configuration previously cost ≈ 90 s.

Table 3. Contact upper bound, one construction call, identical arguments, deadline lifted. The digest hashes every returned placement.

Instance	cells skipped	unsound skips	without (s)	with (s)	result
P3	45.9 M (40.8%)	0	103.52	55.99	identical, 1.85×
P4	11.6 M (3.4%)	0	27.43	16.55	identical, 1.66×
P6	9.3 M (1.3%)	0	102.72	94.61	identical, 1.09×

5 Team Members' Contributions (Not Applicable to Single-Member Teams)

6 AI Use Declaration

Generative AI was used substantially in this work, as an interactive coding and analysis assistant (Anthropic Claude, via its command-line coding interface) throughout development.

Its contributions were: implementing and refactoring the C++ extension modules and the Python scheduler from specifications discussed in dialogue; deriving the contact upper bound of Section 2.2 and its exclusion conditions; writing the measurement harnesses, including the per-cell soundness check and the identical-work replay used for Table 3; running experiments and tabulating results; diagnosing defects, including the position-dependence of the geometric predicate and a double-charged time budget; and drafting this report.

Algorithmic direction, problem interpretation, prioritisation between competing ideas, and the decision of what to accept or reject at each step remained with the authors. Every experimental number reported here was produced by executing the submitted code on the challenge instances; no result in this report was generated or estimated by a language model.